

Լաբորատոր Աշխատանք 4

6. Պահեստի ապրանքների կառավարում

Աշխատանքի նպատակը

Սովորել օգտագործել `struct` կառուցվածքային տիպի զանգվածներ և դրանց մշակման համար ստեղծել առանձին `string` տիպի դաշտերի հետ, մուտքագրել, արտածել և մշակել կառուցվածքների զանգվածի տվյալները: Ամրապնդել զանգվածների դասավորման, որոնման և հաշվարկման հմտությունները:

Աշխատանքի պահանջը

Ստեղծել `Product` կառուցվածք՝ `name`, `category`, `manufacturer`, `price` և `quantity` դաշտերով: Մուտքագրել 12 ապրանքի տվյալներ: Կառուցվածքների զանգվածի մշակման համար ստեղծել առանձին `string` տիպի դաշտեր:

Առաջադրանք

Մուտքագրել 12 ապրանքի տվյալներ: Ֆունկցիաների միջոցով՝

- հաշվել յուրաքանչյուր ապրանքի պահեստային ընդհանուր արժեքը,
- գտնել ամենամեծ ընդհանուր արժեք ունեցող ապրանքը,
- հաշվել ամբողջ պահեստի ընդհանուր արժեքը,
- արտածել տրված կատեգորիայի այն ապրանքները, որոնց քանակը փոքր է տրված սահմանաչափից,
- դասավորել ապրանքները ըստ ընդհանուր արժեքի նվազման կարգով:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
struct Product
```

```
{
```

```
    string name;
```

```
    string category;
```

```
    string manufacturer;
```

```
    double price;
```

```
    int quantity;
```

```

};

// Մուտքագրում
void inputProducts(Product products[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << "\nProduct " << i + 1 << endl;

        cout << "Name: ";

        getline(cin, products[i].name);

        cout << "Category: ";

        getline(cin, products[i].category);

        cout << "Manufacturer: ";

        getline(cin, products[i].manufacturer);

        cout << "Price: ";

        cin >> products[i].price;

        cout << "Quantity: ";

        cin >> products[i].quantity;

        cin.ignore(1000, '\n');
    }
}

// Մեկ ապրանքի ընդհանուր արժեքը
double getTotalValue(const Product& product)
{
    return product.price * product.quantity;
}

// Բոլոր ապրանքների արտաձույթ

```

```

void printProducts(const Product products[], int n)
{
    cout << "\n===== ALL PRODUCTS =====\n";
    for (int i = 0; i < n; i++)
    {
        cout << "Name: " << products[i].name << endl;
        cout << "Category: " << products[i].category << endl;
        cout << "Manufacturer: " << products[i].manufacturer << endl;
        cout << "Price: " << products[i].price << endl;
        cout << "Quantity: " << products[i].quantity << endl;
        cout << "Total value: " << getTotalValue(products[i]) << endl;
        cout << "-----" << endl;
    }
}

// Ամենամեծ ընդհանուր արժեք ունեցող ապրանքը
int findMaxValueProduct(const Product products[], int n)
{
    int max = 0;
    for (int i = 1; i < n; i++)
    {
        if (getTotalValue(products[i]) > getTotalValue(products[max]))
        {
            max = i;
        }
    }
    return max;
}

```

```
}
```

```
// Ամբողջ պահեստի ընդհանուր արժեքը
```

```
double calculateWarehouseValue(const Product products[], int n)
```

```
{
```

```
    double total = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        total += getTotalValue(products[i]);
```

```
    }
```

```
    return total;
```

```
}
```

```
// Տրված կատեգորիայի և սահմանաչափից փոքր քանակով ապրանքների արտածում
```

```
void printProductsByCategory(
```

```
    const Product products[],
```

```
    int n,
```

```
    const string& category,
```

```
    int limit)
```

```
{
```

```
    cout << "\n===== FILTERED PRODUCTS =====\n";
```

```
    bool found = false;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (products[i].category == category &&
```

```
            products[i].quantity < limit)
```

```
        {
```

```

        cout << "Name: " << products[i].name << endl;

        cout << "Category: " << products[i].category << endl;

        cout << "Manufacturer: " << products[i].manufacturer << endl;

        cout << "Price: " << products[i].price << endl;

        cout << "Quantity: " << products[i].quantity << endl;

        cout << "-----" << endl;

        found = true;
    }
}

if (!found)
{
    cout << "No products found." << endl;
}
}

// Դասավորում ըստ ընդհանուր արժեքի նվազման կարգով
void sortByTotalValue(Product products[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (getTotalValue(products[j]) >
                getTotalValue(products[i]))
            {
                Product temp = products[i];
                products[i] = products[j];
            }
        }
    }
}

```

```

        products[j] = temp;
    }
}
}
}

// Դասավորված ապրանքների արտածում
void printSortedProducts(const Product products[], int n)
{
    cout << "\n===== SORTED PRODUCTS =====\n";
    for (int i = 0; i < n; i++)
    {
        cout << i + 1 << ". "
            << products[i].name
            << " | Total value: "
            << getTotalValue(products[i])
            << endl;
    }
}

int main()
{
    const int N = 12;
    Product products[N];
    inputProducts(products, N);
    printProducts(products, N);
    int maxProduct = findMaxValueProduct(products, N);

```

```

cout << "\n===== MAXIMUM VALUE PRODUCT =====\n";

cout << "Name: " << products[maxProduct].name << endl;

cout << "Category: " << products[maxProduct].category << endl;

cout << "Manufacturer: " << products[maxProduct].manufacturer << endl;

cout << "Price: " << products[maxProduct].price << endl;

cout << "Quantity: " << products[maxProduct].quantity << endl;

cout << "Total value: "

    << getTotalValue(products[maxProduct])

    << endl;

cout << "\n===== WAREHOUSE VALUE =====\n";

cout << "Total warehouse value: "

    << calculateWarehouseValue(products, N)

    << endl;

string category;

int limit;

cout << "\nEnter category: ";

getline(cin, category);


cout << "Enter quantity limit: ";

cin >> limit;

printProductsByCategory(products, N, category, limit);

sortByTotalValue(products, N);

printSortedProducts(products, N);

return 0;

}

```

Օրինակային մոդուլ

Product 1

Name: Laptop

Category: Electronics

Manufacturer: Lenovo

Price: 450

Quantity: 5

Product 2

Name: Mouse

Category: Electronics

Manufacturer: Logitech

Price: 25

Quantity: 10

Product 3

Name: Keyboard

Category: Electronics

Manufacturer: Logitech

Price: 45

Quantity: 4

Product 4

Name: Chair

Category: Furniture

Manufacturer: IKEA

Price: 120

Quantity: 6

Product 5

Name: Desk

Category: Furniture

Manufacturer: IKEA

Price: 250

Quantity: 3

Product 6

Name: Notebook

Category: Stationery

Manufacturer: Oxford

Price: 5

Quantity: 20

Product 7

Name: Pen

Category: Stationery

Manufacturer: Parker

Price: 10

Quantity: 15

Product 8

Name: Monitor

Category: Electronics

Manufacturer: Samsung

Price: 300

Quantity: 4

Product 9

Name: Printer

Category: Electronics

Manufacturer: HP

Price: 220

Quantity: 2

Product 10

Name: Shelf

Category: Furniture

Manufacturer: IKEA

Price: 180

Quantity: 5

Product 11

Name: Headphones

Category: Electronics

Manufacturer: Sony

Price: 80

Quantity: 3

Product 12

Name: Backpack

Category: Accessories

Manufacturer: Nike

Price: 60

Quantity: 7

Այնուհետև՝

Enter category: Electronics

Enter quantity limit: 5

Օրինակային ելք

===== MAXIMUM VALUE PRODUCT =====

Name: Laptop

Category: Electronics

Manufacturer: Lenovo

Price: 450

Quantity: 5

Total value: 2250

===== WAREHOUSE VALUE =====

Total warehouse value: 6700

===== FILTERED PRODUCTS =====

Name: Keyboard

Category: Electronics

Manufacturer: Logitech

Price: 45

Quantity: 4

Name: Monitor

Category: Electronics

Manufacturer: Samsung

Price: 300

Quantity: 4

Name: Printer

Category: Electronics

Manufacturer: HP

Price: 220

Quantity: 2

Name: Headphones

Category: Electronics

Manufacturer: Sony

Price: 80

Quantity: 3

===== SORTED PRODUCTS =====

1. Laptop | Total value: 2250
2. Monitor | Total value: 1200
3. Chair | Total value: 720
4. Shelf | Total value: 900
5. Desk | Total value: 750
- ...

Աշխատանքի ակնկալվող արդյունքը

Աշխատանքի արդյունքում պետք է կարողանալ ստեղծել Product կառուցվածք և դրա հիման վրա կառուցվածքների զանգված, մուտքագրել և մշակել ապրանքների տվյալները առանձին ֆունկցիաների միջոցով: Ծրագիրը պետք է կարողանա հաշվարկել յուրաքանչյուր ապրանքի և ամբողջ պահեստի արժեքը, գտնել առավել մեծ արժեք ունեցող ապրանքը, կատարել տվյալների ֆիլտրում ըստ կատեգորիայի և քանակի, ինչպես նաև դասավորել ապրանքները ըստ ընդհանուր արժեքի նվազման կարգով: